Understand | Explore | Evaluate | Make

CHAPTER 14

Activities to Understand the Problem

In the understand mindset we actively seek information from stakeholders and work to define (or redefine) the problem. Understanding is more than just specifying requirements. We also need to figure out who our stakeholders are, identify business goals for the system, and ensure requirements specified with an eye toward the architecture.

As you'll recall from Chapter 5, *Dig for Architecturally*, on page 49, there are four kinds of architecturally significant requirements. All of these requirements will influence the architecture, but quality attributes are the most influential and a key concern for architects.

Constraints Unchangeable design decisions, usually given but sometimes chosen

Quality Attributes Externally visible properties that characterize how the system operates in a specific context

Influential Functional Requirements Features and functions that require special attention in the architecture

Other Influencers Time, knowledge, experience, skills, office politics, your own geeky biases, and all the other stuff that sways your decision making

The activities in this chapter help teams empathize with stakeholders and dig for architecturally significant requirements. Use them when you need to get a better grasp on the real problem.

Activity 1

Choose One Thing

Discuss priorities with stakeholders by presenting them with an extreme choice: if you only get one thing, what will it be? This activity can help stakeholders make decisions when facing difficult trade-offs.

Benefits

- Clearly communicates, *this is more important than that.*
- Starts a conversation about why a certain choice was made and what would need to change for stakeholders to change their mind.
- Makes it obvious when stakeholders disagree.

Participants

All stakeholders

Preparation and Materials

- List of alternatives, such as quality attributes, or other difficult trade-offs, such as cost, schedule, and features.

Steps

1. Explain the rules of the game. Stakeholders may choose only one of the presented options. Remind participants that this does not mean you will do only one thing, but instead the point is to have tough conversations early to avoid trouble down the road.
2. Present a set of options to stakeholders. Discuss what each option means and make sure everyone understands the options.
3. Force stakeholders to pick one option. All stakeholders should agree that this is the most important thing. In this activity we want consensus.

4. Briefly discuss why the option was selected. This discussion is often more important than the option picked.
5. Pick another set of architecturally significant requirements that are in tension with one another and play again.

Guidelines and Hints

- Play this game before things get too difficult. This conversation is easier when it's hypothetical than when it's real.
- Quality attributes that are in tension with one another should be pitted against one another.
- Use this activity to prioritize influential functional requirements.
- This technique works well as an informal conversation to help get better understanding of stakeholders' true need.

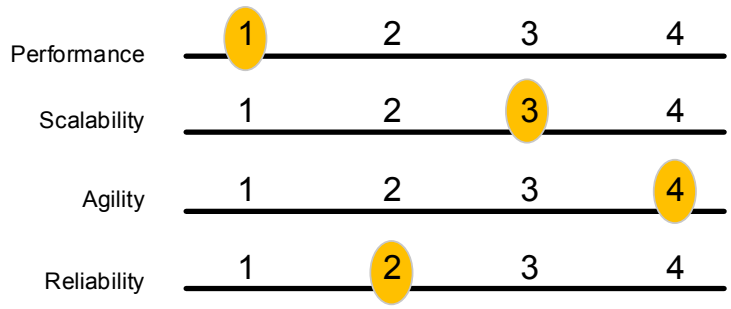
Example

Here are a few scenarios one team posed to some stakeholders and how things played out when stakeholders were asked to choose one thing:

Matchup	Stakeholder's Choice
Faster performance or greater accuracy	Faster performance, assuming accuracy met a required minimum threshold.
Cost vs. time-to-market	Time-to-market; stakeholders were willing to accept greater technical debt to get required features by a specific date.
Usability vs. security	Security; this was the number one quality attribute and surprisingly beat several other important quality attributes.
Availability vs. cost	Availability; achieving high availability on this particular system required potentially expensive redundancy for which stakeholders would willingly pay (up to a point) if required.

Alternatives

Instead of pitting one alternative against another, use *trade-off sliders* to let stakeholders make multiple comparisons. To use this activity, identify 3–5 related alternatives. Each item can receive a number 1 – N, where N is the number of items in the list. No two items can have the same number. This activity is often presented visually using sliders.



Activity 2

Empathy Map

Brainstorm and record a particular stakeholder's responsibilities, thoughts, and feelings to help the team develop a greater sense of empathy with stakeholders' goals.

Benefits

- Discover your audience's needs before developing an architecture description
- Help decide what information to include or exclude
- Create a rubric for evaluating the effectiveness of an architecture description

Activity Timing

10–30 minutes

Participants

Software architect, development team

Small groups of 3–5 or as a solo exercise

Preparation and Materials

- Before the activity starts, choose which stakeholders, systems, or users will be the focus of the activity.
- Flipchart paper or a whiteboard, markers, and sticky notes
- This activity can be adapted for remote participants with screen-sharing or remote collaboration software.

Steps

1. Draw a grid on a whiteboard or piece of paper. Label each quadrant—do, make, say, and think.
2. Pick a specific stakeholder and write his or her name in the middle.

3. Brainstorm tasks this person does, artifacts this person makes, things this person says, and feelings this person may have.
4. Write each idea on a sticky note and place it in the corresponding quadrant.
5. Review the empathy map and highlight insights.

Guidelines and Hints

- Be specific. Pick a person to empathize with, not a general role.
- Validate the findings from your empathy map with stakeholders.
- Mention quality attributes, risks, or other concerns this person may find relevant.
- Adapt this method for understanding application end users, external systems (for interface design), or for use with proxy stakeholders to understand quality attributes.
- Use software such as Mural[1] when participants are distributed.

Example

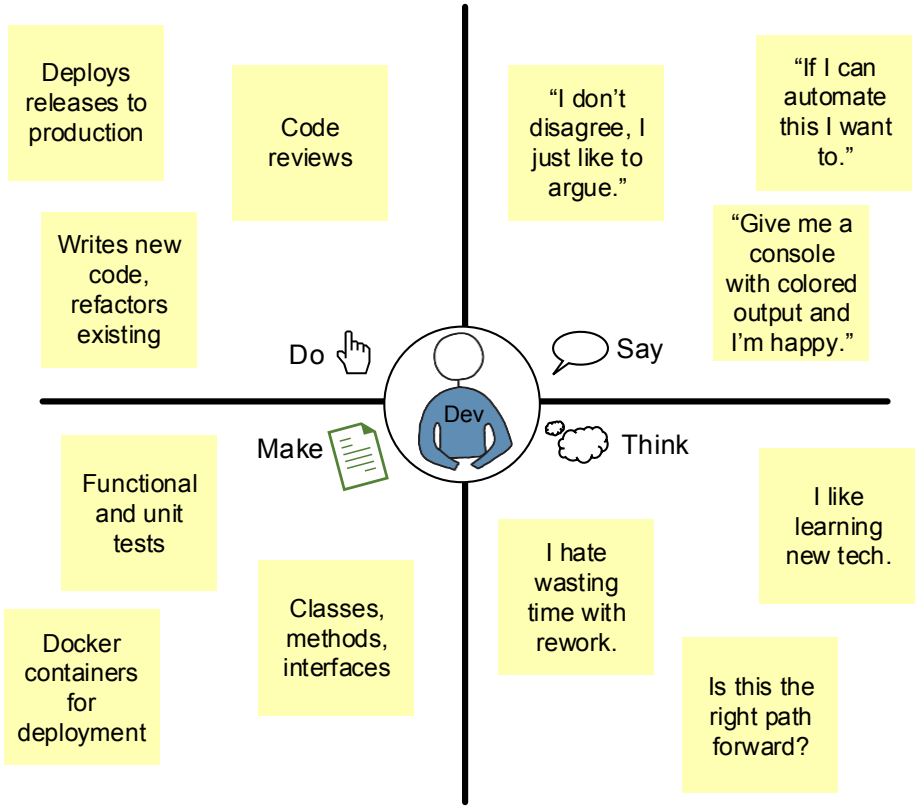
There is an example empathy map for a developer stakeholder persona on page 197.

Alternatives

The quadrants of an empathy map can be changed. Another common schema is hear, see, do (or say), and think (or feel).

Empathy maps are also useful for quality attribute analysis.[2] This approach is especially useful when your stakeholders are unable to participate in other workshops, such as the Activity 7, *Mini-Quality Attribute Workshop*, on page 210. Instead of focusing on what the stakeholders do, say, or think, we focus on how stakeholders react to specific quality attributes. Obviously, it's better to ask stakeholders directly. When they are not available, this is a good substitute.

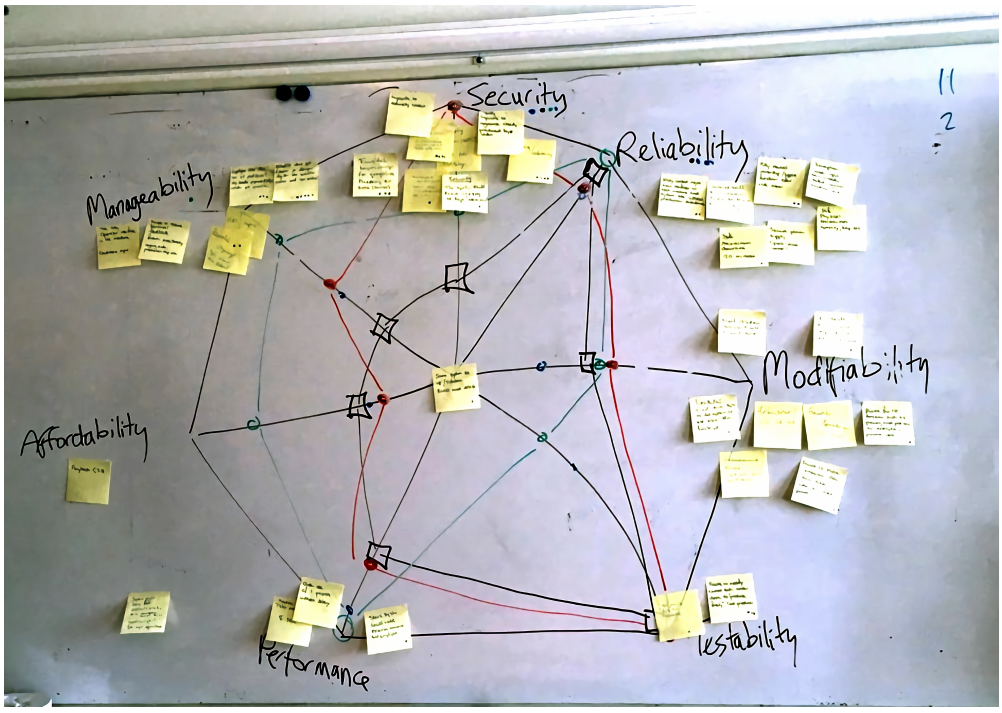
1. https://mural.co/
2. Thijmen de Gooijer. *Quality Requirements on a Shoestring*. SATURN 2015. http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=436426



To use this variant, pick a stakeholder and brainstorm at least two quality attribute scenarios or general concerns for each relevant quality attribute. Use dot voting[3] to estimate how this stakeholder might rate the quality attributes. Ideally, you should validate the outcomes of this exercise, but this is not always practical. These insights can be used during other workshops to help ensure a stakeholder's perspective is represented even when that stakeholder is not present.

3. http://dotmocracy.org/dot-voting

In the following example, from a workshop facilitated by Thijmen de Gooijer, sticky notes represent raw quality attribute scenarios. The lines in the center of the chart show how different absentee stakeholders might have felt about different quality attributes shown on a quality attribute web on page 207. The quality attribute web helps us visualize the importance of different quality attributes during structured brainstorming activities. During the workshop, different participants were asked to play the role of an absentee stakeholder by using the empathy map as a guide.



Activity 3

Goal-Question-Metric (GQM) Workshop

An approach for identifying metrics and response measures so that we can connect data with business goals. The *Goal-Question-Metric approach* (GQM) was introduced by Victory Basili, Gianluigi Caldiera, and H. Dieter Rombach in *The Goal Question Metric (GQM) Approach [BCR94]*. The goal of GQM is to identify measures we can use to determine whether a goal has is satisfied.

There are three parts to the GQM approach. The goal defines conceptual requirement that must be met. Goals can describe quality attribute scenarios, general software quality, business goals, or other topics. Questions illustrate the means by which we can characterize one or more goals. Metrics defines the measures needed to answer one or more questions.

Benefits

- Emphasizes using stakeholder goals as the basis for measures
- Shows clear lineage from data to stakeholder goals by way of the questions that must be answered to decide whether a goal is satisfied
- Flexible approach that can help teams think about metrics in a variety of situations

Activity Timing

15–90 minutes

Participants

This activity can be done solo or in a small group of 2–5 people. Any mix of stakeholders will work.

Preparation and Materials

- Whiteboard or flipchart papers, markers
- Goals to be explored can be optionally identified before the workshop

Steps

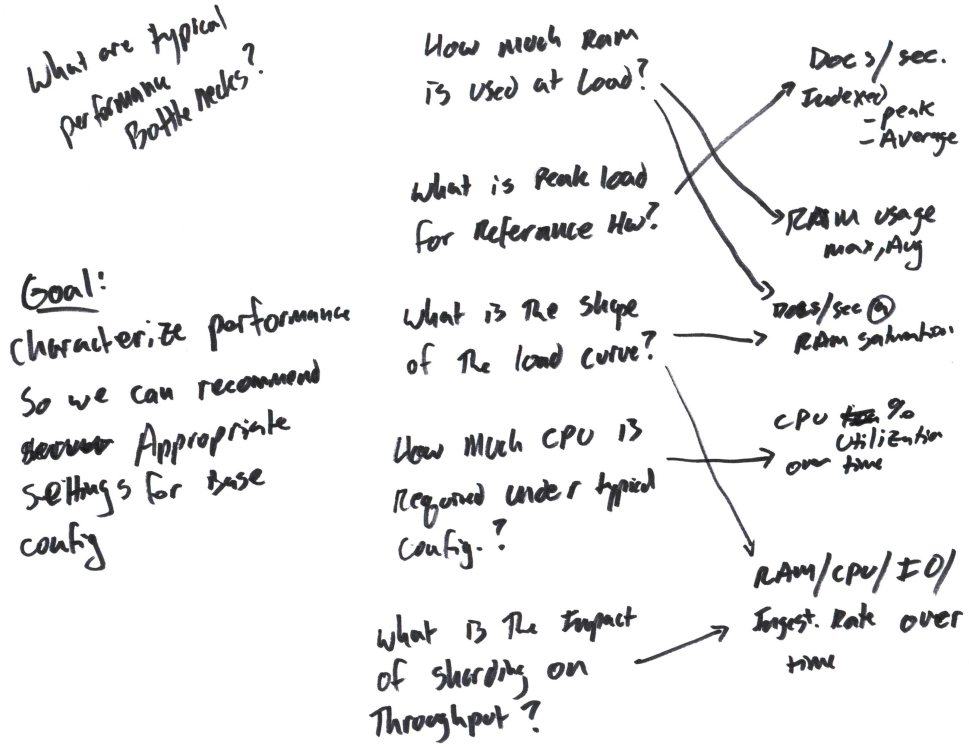
1. Write a goal at the far left of the whiteboard.
2. Prompt participants to provide questions. *What questions would you need to answer to know if we've met this goal?* Write each question to the right of the goal. Draw lines from the goal to the questions to create a tree.
3. Explore each question to identify metrics needed to answer the question. Write each metric to the right of the questions. Draw lines from each question to the metrics required to answer it.
4. Repeat the exercise for any related goals that might reasonably use the same questions or metrics. The end result should be a tree that connects metrics to questions and questions to goals.
5. Identify data required to compute each metric. Write the data needs to the right of the metrics. Draw lines from each metric to the data needed to compute the metric.
6. For each piece of data identified, determine where you can get the data. Write down each data source for each bit of data. Describe the cost of gathering data from each of the data sources.
7. The last phase of the workshop is to prioritize data and metrics. Clearly identify *must have* metrics. Look for data sources that can provide data needed to compute multiple metrics or metrics that can answer multiple questions.
8. Record the results of the workshop by taking pictures and writing down the discussed goals, questions, metrics, data, and data sources.

Guidelines and Hints

- Be sure to have plenty of space for drawing the GQM tree.
- Look for opportunities for reuse. Metrics can be used to answer more than one question. Likewise, the same data might help compute multiple metrics.
- Data sources and data are likely to influence the architecture, but also look for opportunities to gather data outside of the architecture.
- Record results in a table or spreadsheet to validate with stakeholders later. The identified metrics are revisited throughout the life of the software system.

Example

In this example, the goal is written on the far left, questions are captured in the middle column, and metrics are written in the column on the far right. Data used to compute the metrics was omitted from this particular GQM sketch.



Activity 4

Interview Stakeholders

Sometimes the easiest way to learn about stakeholders' business goals is to ask. When we *interview stakeholders* we ask about their plans for the software, uncover the problem context, get a feeling for looming risks, and dig for details about quality attributes and other requirements.

Interviews may be either structured and follow a set script or unstructured. Unstructured interviews are more conversational and often easier for the subject, but you should still go into the interview with a planned set of topics. Interviews can also be either face-to-face or conducted offline via questionnaires or surveys.

There are many stakeholder interview resources with checklists and question templates. I recommend *Designing for the Digital Age: How to Create Human-Centered Products and Services [Goo09]* by Kim Goodwin if you are looking for further depth regarding this technique. An extensive excerpt from the chapter on stakeholder interviews is freely available on the web.[4]

Benefits

- Focus on general information gathering
- Format allows for open back-and-forth discussion
- Provides background information that can be used to prepare for other workshops or activities
- Quickly validate quality attribute scenarios and other ASRs
- Creates a direct connection between stakeholders and architects

Activity Timing

A single interview should last no more than 30–60 minutes.

Participants

This activity can be done one-on-one or in small stakeholder groups. The architect leads the interview. Stakeholders are the interview subjects.

4. http://boxesandarrows.wpengine.com/understanding-the-business/

Additional team members may observe an interview, but the interview group should be small—no more than one or two active interviewers—to avoid overwhelming the subject.

Preparation and Materials

- Interview goals and questions
- Pen and paper or laptop for taking minor notes during the interview
- Voice recorder to record the session so you can focus on the conversation. Write detailed notes after the interview. Most teleconference software has options for recording.

Steps

1. Explain the goals of the interview and how the results of the interview will be used. *We're going to validate some requirements we've gathered so far to make sure we capture your real needs.*
2. Ask the subject questions from your planned interview checklist.
3. Follow up with clarifying questions as needed to be sure you get the information you need.
4. Conclude the interview by thanking the subject for taking time to meet with you.
5. Directly after the meeting, jot down your general impressions of the interview, including any themes, technical asides, and design thoughts. If others observed the interview, collect their notes and general impressions.
6. Once all interviews have been completed, analyze the data collected. Update or create architecturally significant requirements as appropriate. Summarize any new risks or concerns that may require further action.
7. Once all interviews have been completed, hold a debrief meeting with the team and stakeholders to share insights.

Guidelines and Hints

- Avoid interviewing stakeholders about architectural concerns too early. There is often a significant amount of general design work that must happen before diving into architectural concerns.
- Phrase questions so the subject can share their true thoughts. Avoid leading the subject.
- When possible, use the subject's words when summarizing ideas.

- Talk to real users and primary stakeholders of the system being designed. For example, it's better to interview Eunice, who trains the hamsters herself, than Beatrice, who only oversees hamster trainers but doesn't train the little critters.
- Use data to help jump start conversations. See Activity 9, *Response Measure Straw Man*, on page 219 for a method that can gently encourage interview subjects to be more specific.
- Record the session or have a designated note taker so the interviewer can devote his or her full attention to the subject.

Example

Here is an example of how an unstructured interview might go. In this exchange, the architect is attempting to clarify a business constraint.

> ***Architect:*** *You mentioned earlier that this new system replaces an existing one. What is the plan for the old system?*
>
> ***Stakeholder:*** *Once the new system is on line we'll start the deprecation time line for the old system. The deprecation process can take up to nine months since we have to give current customers time to migrate off the old system.*
>
> ***Architect:*** *Nine months is a long time. When do you hope that process will complete?*
>
> ***Stakeholder:*** *The earlier the better, but I'm hoping by December of next year.*
>
> ***Architect:*** *OK, working backward, to be able to deprecate by December the new system needs to be live by the end of March. Does that sound right to you?*
>
> ***Stakeholder:*** *That's probably about right, yes.*
>
> ***Architect:*** *Can you tell me a little more about the deprecation requirements? I want to verify that we aren't missing anything that could put deprecation at risk.*
>
> ***Stakeholder:*** *Sure, there are four must-have features we need in the new system before we can deprecate the old one…*

With that last statement, the architect immediately begins thinking about influential functional requirements.

Activity 5

List Assumptions

Assumptions are truths about the system we simply take for granted. Hidden assumptions kill projects (or at least cause significant pain). With the *list assumptions* activity, we take assumptions out of the shadows by writing down as many assumptions as we can. Use this information to plan further design work, prioritize next steps, improve ASRs, and improve the team's shared knowledge about the architecture.

Benefits

- Head off misunderstandings about the true goals and requirements.
- Great for ad hoc analysis. Does not require a formal workshop or agenda.
- Avoid missing important requirements.

Activity Timing

15–30 minutes

Participants

Whole team working in pairs or small groups (no more than 3–5 people).

This can be done as a solo exercise, but you would need to share your assumptions list with someone; otherwise, your assumptions will remain hidden!

Preparation and Materials

- Any writing surface and writing tool: pen and paper, marker and whiteboard, or sharpie and sticky notes

Steps

1. Kick off the activity with the adage *You know what happens when we assume too much, right? It makes an ass out of you and me!*
2. Explain the goal of the activity. *Over the next 15 minutes we're going to write down all the assumptions we have about the system.*
3. Prompt participants by focusing on an area where assumptions need to be flushed into the open.

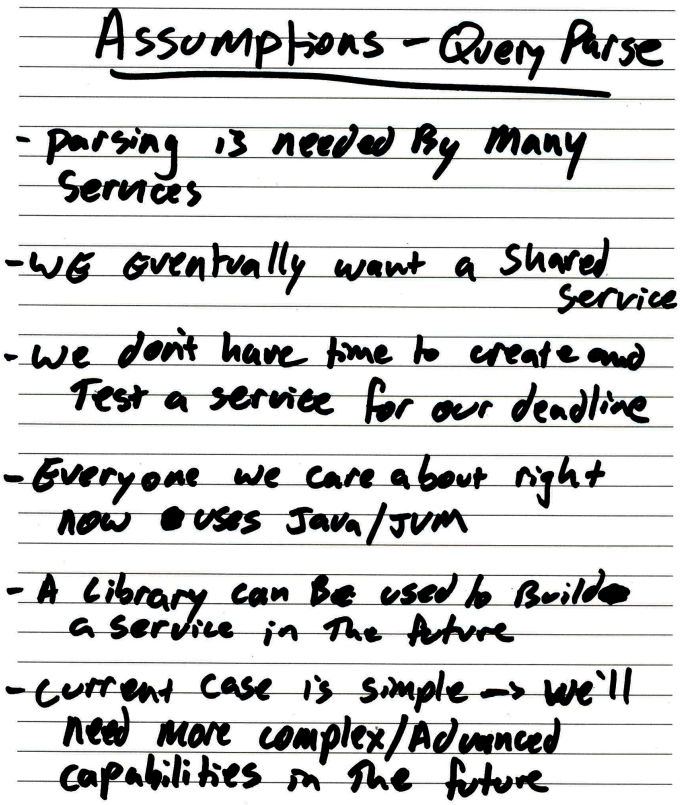
4. Write down the assumptions mentioned so everyone can see them.
5. As the pace slows, move on to the next topic or end the session by planning follow-up actions.

Guidelines and Hints

- Start by asking, *what do we think we know about X?*
- Write down everything mentioned, even if it seems like common knowledge.
- Pause to discuss assumptions that are surprising or generate a reaction from one or more participants.
- Record the assumptions in your team wiki.

Example

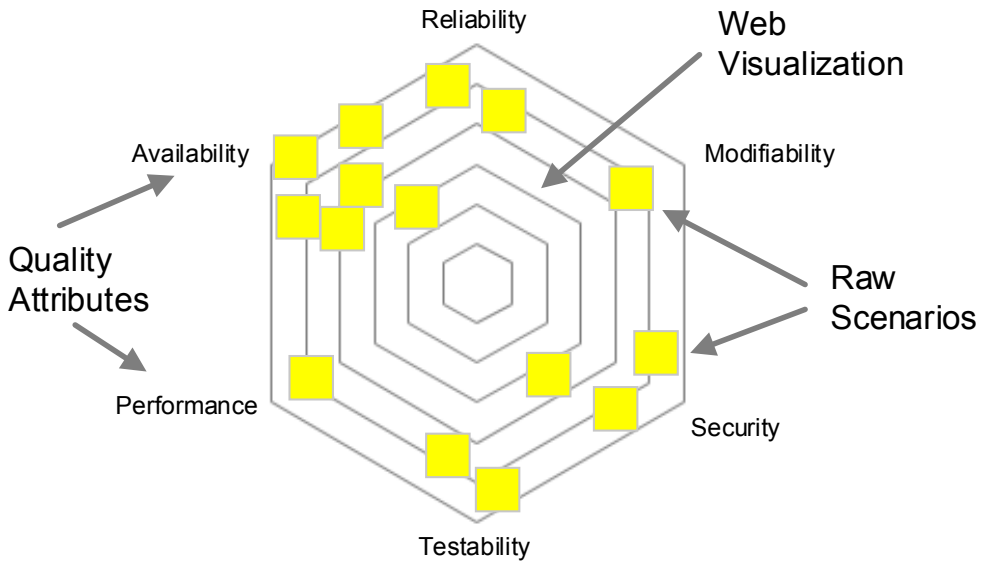
Here is an assumptions list drawn up immediately following Activity 19, *Whiteboard Jam*, on page 255:



Activity 6

Quality Attribute Web

The *quality attribute web* is a brainstorming and visualization activity to help elicit, categorize, refine, and prioritize stakeholder concerns and raw quality attribute scenarios. A quality attribute web captures stakeholders' concerns. We write each concern on individual sticky notes. The web is drawn as a simple radar chart with relevant quality attributes written around the edge like this:



Benefits

- Guide stakeholders to think about quality attributes instead of features.
- Provide a visualization that shows how one system is different from another based on highly desirable properties.
- Help stakeholders prioritize quality attribute scenarios before refining them.

Activity Timing

30–45 minutes

Participants

Any stakeholders, including the team

Preparation and Materials

- If you are using a quality attribute taxonomy, prepare it ahead of time. You may find it helpful to print the web on poster paper instead of drawing it on a whiteboard.
- Sticky notes, markers

Steps

1. Draw or post a blank quality attribute web so everyone can see it. The web can be created ahead of time if you know which quality attributes to include. If you're not using a prepared web, brainstorm as a group to identify 5–7 quality attributes that are important to the stakeholders.
2. Brainstorm concerns and raw quality attribute scenarios as a group. Write each concern down on a sticky note and add it to the web near the quality attribute to which it most closely applies.
3. When time expires, write down the concerns and use the information to create quality attribute scenarios.

Guidelines and Hints

- Some stakeholders will need help getting started. Be prepared to help them phrase their concerns initially.
- Use dot voting to prioritize concerns on the web.
- Don't worry about getting perfect scenarios. A general thought, worry, response measure, or partial scenario is a great start.
- Combine with the mini-quality attributes workshop, described on page 210, for a more comprehensive workshop.

Example

In this example, quality attributes were brainstormed when the activity began and written on a whiteboard. In this particular workshop, you can see that availability and reliability tended to be on everyone's minds slightly more than other quality attributes. Of the twenty or so raw scenarios created during the hour long activity, only six or seven were prioritized highly by stakeholders. The remainder helped the team gain necessary context about the stakeholders' concerns.

Activity 7

Mini-Quality Attribute Workshop

The *mini-Quality Attribute Workshop* (mini-QAW) is a lean, facilitated workshop designed to help you talk about quality attributes with stakeholders early in a system's life.[5] During a mini-QAW, you'll collaborate as a group to quickly identify, develop, and clarify quality attributes with the help of a quality attribute taxonomy. By the end of the mini-QAW, you'll have a prioritized list of quality attribute scenarios and a wealth of contextual information about the system to be designed.

Benefits

- Walk through the essential steps of a traditional quality attribute workshop in only a few hours.
- Quickly identify raw quality attributes and prioritize them before refining into full scenarios.
- Provide opportunities for stakeholders to riff on each other's ideas.
- Create a forum for open discussion among stakeholders to discuss quality attribute concerns, risks, and other general concerns about the software system.

Activity Timing

Ninety minutes to 3 hours, depending on the size of the taxonomy and brainstorming method used

Participants

A facilitator, usually the software architect. A small group of stakeholder participants.

This workshop works best in small groups of 3–5, with a maximum size of about 10 participants. Host multiple workshops if necessary to keep the group size down. When hosting multiple workshops, review scenarios with all groups once the workshops have concluded.

5. http://bit.ly/mini-qaw

Preparation and Materials

- Before the workshop, prepare a *quality attribute taxonomy*. The quality attribute taxonomy is a set of predefined quality attributes highly relevant to the type of system you are building. An example of a quality attribute taxonomy for service-oriented architectures is available from the Software Engineering Institute.[6] The taxonomy will be used to facilitate structured brainstorming.
- Prepare graphical quality attribute scenario templates in the style of the examples on page 53. Use these templates to capture scenarios during the workshop.
- If desired, prepare a quality attribute web, shown on page 207, on poster-sized paper for use during the workshop. If not using a pre-printed taxonomy web, draw a web at the start of the workshop.
- Sticky notes and markers for participants

Steps

1. Present the workshop goals and agenda.
2. Teach participants what they need to know about quality attributes. Describe the quality attribute taxonomy you'll use during the workshop.
3. Display or draw the quality attribute web so everyone can see it.
4. Brainstorm raw quality attribute scenarios using either structured brainstorming or a questionnaire. Instruct participants to write one idea per sticky note and place them directly on the displayed taxonomy web. Read the posted raw scenarios out loud as they are placed on the web. If this prompts participants to think of new scenarios, record and post them on the web too.
5. After the brainstorming phase, prioritize the quality attributes and raw scenarios using dot voting. Participants get 1/3 the number of identified raw scenarios. For example, if there are 25 sticky notes on the web, everyone gets 8 votes to spend however they please. Participants also get 2 votes for overall quality attributes. Everyone votes at the same time.

6. Liam O'Brien, Len Bass, and Paulo Merson. *Quality Attributes and Service-Oriented Architectures*. http://resources.sei.cmu.edu/library/asset-view.cfm?assetid=7405

6. Refine the top raw scenarios as a group until time runs out using the six-part scenario template shown on page 53. Remaining work must be done as homework.
7. As homework, refine the top raw quality attribute scenarios. Present the top refined quality attribute scenarios in a follow-up meeting to verify the scenarios and relative priority.

Guidelines and Hints

- Keep your taxonomy small, 5–7 quality attributes max.
- Use the web visualization to drive the workshop. Put the sticky notes close to related quality attributes.
- Don't worry about creating formal scenarios during brainstorming.
- Ask probing questions about the stimulus, response, environment.
- Pay attention when stakeholders sound worried about something. Stakeholders' worries are often the source of a possible scenario.
- Watch out for features and functional requirements.
- Do not skip the homework. This is the most important part!
- If workshop participants are not co-located, select screen-sharing software all participants can use or consider using a digital whiteboard application such as Mural. See *Work with Remote Teams*, on page 124 for more remote facilitation tips.

Example

Here is an example mini-QAW agenda:

Agenda Item	Timing	Hints
Introduce the Mini-QAW	10 minutes	
Teach participants about Quality Attributes	15 minutes	Set participants up for success
Brainstorm Raw Scenarios	30 minutes–2+ hours	Walk the System Properties Web
Prioritize raw scenarios	5 minutes	Use dot voting
Refine scenarios	Until time runs out	Finish as homework
Review the results	1 hour	Separate, future meeting

The mini-QAW is a very useful workshop with a few moving parts. Next, we'll look at some additional tips for each of the stages in the standard agenda.

Brainstorm and Prioritize Raw Scenarios

If workshop participants are relatively experienced, guide them through a simple brainstorming exercise. Set a time limit of 7–10 minutes for brainstorming and have participants work alone to come up with as many raw scenarios as they can. With less experienced participants (or facilitators), consider using the quality attribute web activity on page 207 with a prepared taxonomy and *quality attribute taxonomy-based questionnaire*. The taxonomy questionnaire is a list of questions based on a predefined quality attribute taxonomy designed to prompt stakeholders to think about potential scenarios. Questionnaires require more up-front work, but this approach is thorough and produces more consistent results than brainstorming without a questionnaire.

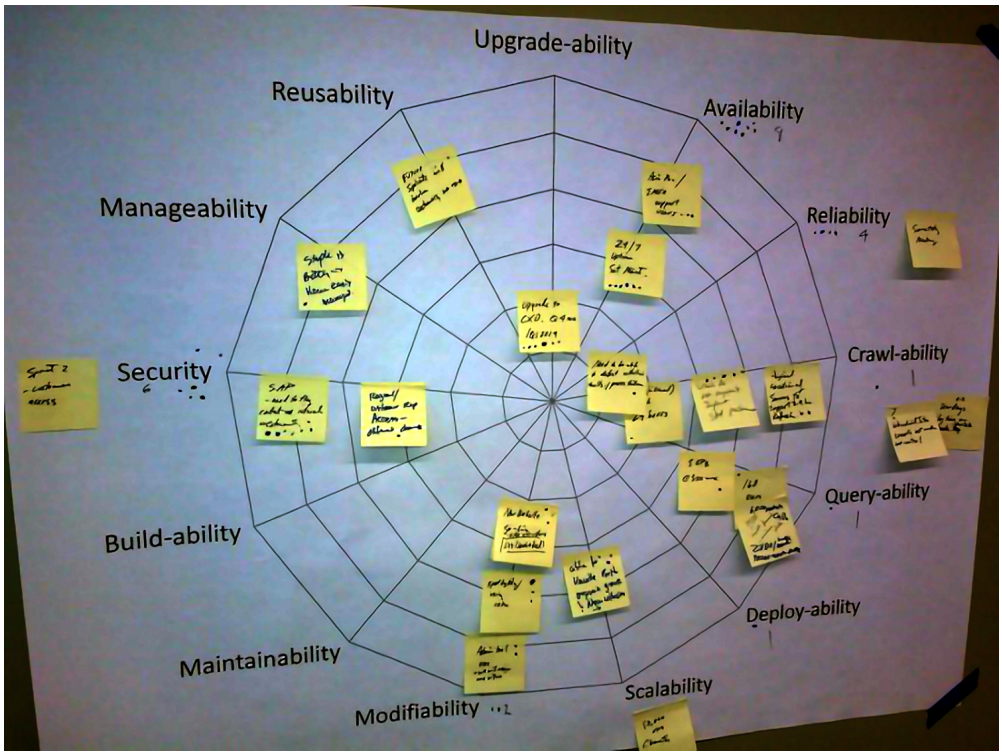
After brainstorming, prioritize the raw scenarios. Stakeholders will raise many concerns during a workshop, but not all concerns are worth the effort to refine further. After participants have finished voting, take a step back and look at the web. Are there areas of the web with a greater number of sticky notes than others? How does that compare with how people voted? Were the high-priority scenarios aligned with the high-priority quality attributes?

The example on page 214, is what the quality attribute web might look like after voting. Dots on sticky notes are a vote for a raw scenario whereas dots on the web are for the overall quality attribute regardless of the specific scenarios identified.

Start Scenario Refinement

After prioritizing raw scenarios, use the time remaining in the workshop to refine scenarios as a group. Show the quality attribute scenario template during the workshop and fill it in with stakeholders. The template can be printed on paper or shown as a presentation. The facilitator is responsible for refining any remaining scenarios as homework before the next meeting.

As you refine scenarios, keep an eye out for functional requirements masquerading as quality attribute scenarios. Everyone loves to talk about features, and it's easy for feature requests to come up during a QAW. When this happens, add the feature request to your notebook and redirect the conversation back toward specific quality attributes.



Verify Findings with Stakeholders

Hold a follow-up meeting to review the refined scenarios with stakeholders. Prepare a slide-based presentation of the findings or other appropriate write-up to share during the meeting.

During this follow-up meeting, check the accuracy of any straw man numbers you put into the scenarios (see Activity 9, *Response Measure Straw Man*, on page 219. Discuss information missing from scenarios and fill what you can. Finally, use this opportunity to double-check the priority of the top quality attribute scenarios. A simple *high* or *low* is usually sufficient. Any raw scenarios not refined are considered as *low* priority.

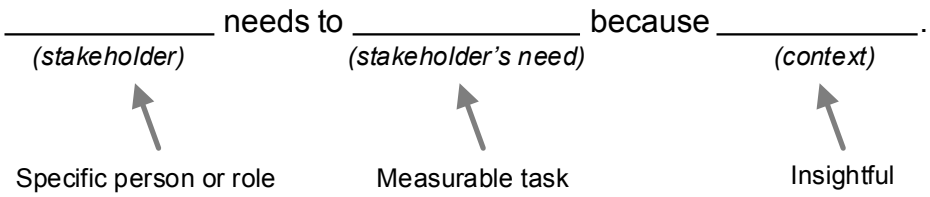
Alternatives

The mini-QAW is based on a more comprehensive workshop. The traditional QAW takes a few days to complete and is more appropriate for high-risk systems with many stakeholders. *Quality Attribute Workshops (QAWs), Third edition [BELS03]* describes the traditional QAW in detail.

Activity 8

Point-of-View Mad Lib

The Point-of-View (POV) mad lib summarizes business goals and other stakeholder needs in a memorable, engaging format. Here's a basic POV mad lib template. You can use this template as is or create your own.



The format should be familiar to anyone who has written agile stories, though the emphasis is on stakeholders' needs and how the overall system will provide value rather than specific features or functionality. You might think of it as a *user odyssey*, a statement that encompasses potentially multiple epics and stories.

Benefits

- Develop empathy for stakeholders' needs.
- Articulate business goals in a user-focused way.
- Use to start the conversation about business goals.

Activity Timing

30–45 minutes

Participants

Any stakeholders. This activity can be done alone or as a small group of 2–3 people. If necessary, a larger group can be divided into smaller subgroups of 2–3 people each.

Preparation and Materials

- Before the activity, identify the list of stakeholders for which you'll produce mad libs. This list can be created just-in-time with participants before introducing the mad lib activity.
- Markers and sticky notes for each group. Enough paper for each group to produce one mad lib per stakeholder.

Steps

1. Introduce the activity by sharing the goal of the exercise.
2. Describe the mad lib template and do a warm-up exercise to ensure participants understand the mad lib format. Everyone should participate in the warm-up.
3. Introduce the first stakeholder. Briefly share any information known about the stakeholder and discuss their needs as a group.
4. Give each group 90 seconds to create a mad lib.
5. Repeat steps 4–5 until all stakeholders have been covered.
6. Share the mad libs produced and briefly discuss as a group. Consensus is not required as a part of this activity.

Guidelines and Hints

- Be specific. Pick an actual person if you can.
- Don't worry about phrasing at first. It can be difficult to find exactly the right words. Getting the ideas out is more important.
- The impact of each mad lib should be outcome focused. Try the *5 Whys* technique[7] to help get to the bottom of stakeholders' real needs.

Example

The POV mad lib is meant to be filled in fast. Don't overthink it. Here are some example mad libs for the Project Lionheart case study:

- Mayor van Damme wants to reduce procurement costs by 30 percent because he wants to avoid cutting funding to education in an election year.

7. https://en.wikipedia.org/wiki/5_Whys

- Mayor van Damme wants to improve city engagement with local businesses because it may improve the local economy when local businesses win contracts.
- The Office of Management and Business wants to cut the time required to publish a new RFP in half because it improves services and reduces costs at the same time.

Alternatives

Any of these approaches, and many others, may be substituted for the Point-of-View mad lib.

Design Hills

Design hills describe the impact stakeholders hope the software will have on end users.[8] Like other ways of specifying business goals, hills try to describe the value the software provides, not how the software is to be built.

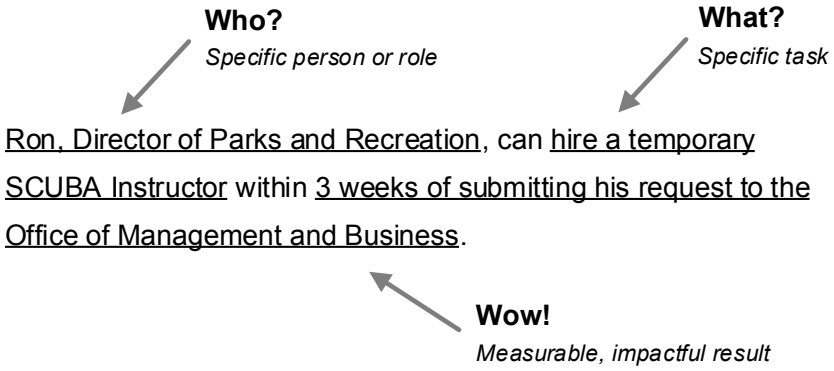
Design hills have three parts: who, what, and wow.

Who? A specific stakeholder who is affected by the software to be built.

What? Something the stakeholder will be able to accomplish with the software that he or she could not do before.

Wow! A significant, measurable outcome that directly results from having used the software to complete the task.

Here's an example from Project Lionheart:



8. http://www.ibm.com/design/thinking/keys/hills/

Traditional Business Goal Statement

Traditional business goal statements are plain and direct statements that describe how stakeholders derive value from the system. Business goal statements have three parts, often enumerated in a table.

Subject A specific person or role.

Outcome A specific and measurable description of how the world changes if the system is successful.

Context Describes the conditions around the goal so the team can develop empathy and a deeper understanding of the need.

Here's an example of the same POV mad libs written as traditional business goals:

Stakeholder	Goal	Context
Mayor van Damme	Reduce procurement costs by 30%	Strong desire to avoid making budget cuts to education in an election year.
Office of Management and Business	Cut the time required to publish a new RFP in half	Current publishing time is 9 weeks. Reducing time improves services across the city and reduces costs at the same time. Citizens suffer when city services go unfunded. Think: *no toilet paper at the girls' basketball game* or *not enough hypodermic needles for emergency medical crews.*

Activity 9

Response Measure Straw Man

The goal of a *response measure straw man* is to give stakeholders something to beat up until they arrive at their own answers. We do this by inventing a reasonable response measure for some quality attribute scenario as a way to kickstart discussions. The straw man technique works with other architecturally significant requirements discussed in Chapter 5, *Dig for Architecturally*, on page 49.

Benefits

- Provides an example of a measurable response and response measure
- Jump-starts thinking about quality attribute scenarios
- Overcomes blank-page syndrome by providing something to edit instead of creating response measures from scratch

Activity Timing

Varies, often combined with other activities

Participants

Architects will often create straw man response measures on their own and validate with stakeholders later.

Preparation and Materials

- A list of raw quality attribute scenarios as described in *Capture Quality Attributes as Scenarios*, on page 52

Steps

1. For each quality attribute scenario, make up a response and response measure. The response should be a reasonable, best guess based on your knowledge and experience. Response measures can be either outrageous or honest.
    - Choose an honest response measure when you think you can confidently estimate a good measure.

- Choose an outrageous response measure when your confidence is low to help find the boundaries around acceptable behavior.

2. Label the scenarios as having a *straw man response measure* to avoid potential future confusion.
3. Validate the scenarios and their response measures with stakeholders, such as during a stakeholder interview, described on page 202, or mini-QAW, described on page 210.

Guidelines and Hints

- Use a straw man to understand the boundaries around acceptable behavior.
- Responses should be correct for the scenario. The point is to zero in on an accurate and reasonable response measure.
- Listen to your stakeholders once you get them talking. When presented with a wrong answer, many stakeholders will react with useful information.
- Keep an eye out for anchoring. Anchoring is a cognitive bias where people let the first information they hear drive their decision making. The straw man should be a reasonable estimate or so outrageous it will be rejected outright. Exercise caution if your outrageous estimate is accepted.

Example

Here are some examples of response measure straw men created for a cloud-based information system:

Quality Attribute	Response	Straw Man Response Measure	Accepted Response Measure
Changeability	Time required to add a new algorithm	6 months	2 iterations
Portability	Effort required to move to new cloud provider	3 person-months	4 person-days
Performance	Average response time under typical load	1 minute	3 seconds max
Scalability	User load the system should be able to handle	10 requests per second	140 requests per second

Activity 10

Stakeholder Map

A stakeholder map is a network diagram of the people involved with or impacted by the software system. Use this method to visualize the relationships, hierarchies, and interactions between all the people who have an interest in the software system to be built.

Benefits

- Identify more stakeholders than just the usual suspects.
- Determine who to talk to about requirements.
- Help the team empathize with people and not just focus on technology.
- Create a snapshot of the system context and who's involved.
- Use as a document to bring new teammates up to speed or to assist with architecture validation.

Activity Timing

30–45 minutes

Participants

Whole team, known stakeholders

This activity can be conducted alone or with groups of 25 or more people depending how much physical space is available.

Preparation and Materials

- A drawing surface such as a large whiteboard or large sheets of paper. Tape paper to a wall or roll out on a large table. Provide markers of different colors so that most participants have a marker. When working with a group, be sure there is enough space and writing surface for all participants to contribute.
- If the participants are distributed, consider using a tool such as Mural.[9]

9. https://mural.co/

Steps

1. Introduce the activity by sharing the goal of the exercise. You might start by saying, *For the next 30 minutes we're going to explore who our stakeholders are. Once we have a better idea of who has a stake, we'll come up with a plan for who we're talking to first.*
2. Share the guidelines and hints for creating a stakeholder map.
3. Start the activity. Working together, everyone adds and annotates stakeholders collaboratively until time runs out or the map seems complete.
4. Once the map is complete, ask participants to share observations about the map. Are there interesting connections or unexpected stakeholders? Who are the *most important* stakeholders?
5. Take a picture of the map and store it in your team's wiki.

Guidelines and Hints

- Use simple icons to represent individual people; use multiple icons to represent groups.
- Be specific when naming stakeholders. Think about their roles or in some cases specific names.
- Use speech bubbles to represent stakeholders' needs or thoughts.
- Connect people using arrows to show relationships and influence. Label connections to describe relationships.
- Encourage participants to look beyond the obvious stakeholders if they stall out during the activity.
- Nudge wallflowers, participants who are just watching, to pick up a marker and add to the map.

Example

Here's an example stakeholder map created by three people in about 15 minutes: